Data Engineering & AI Engineering

یادداشتهای آموزشی، مفاهیم فنی و تمرینهای دوره



پیش‌نیازهای مهندسی داده (۱): گیت، لینوکس و مفاهیم پایه شبکه

جلسه سوم
مدرس :احمد کریمی
تاریخ :یکشنبه ۲۷ مهر ۱۴۰۵

تاریخ به روز رسانی : 20261019

این جزوه با هدف ثبت، سازماندهی و بازنویسی آموزشی مطالب دوره رایگان »مهندسی داده و مهندسی هوش مصنوعی« تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثالهای مطرحشده در کلاس و پرسشوپاسخهای آموزشی تدوین میشود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبتهای مدرس تغییر کند.

هدف این جزوه صرفاً خلاصهکردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آمادهشدن برای فعالیتهای عملی به آن مراجعه کند.

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده میشوند. سپس مثالهای مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی میشوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- مطالب اصلی جلسه :بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثالهای اصلی.

تکمیل فنی :توضیحات دقیقتر، اصلاح برداشتهای احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.

- مثال اضافی :مثالها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شدهاند.
- نکات کاربردی :ملاحظات اجرایی، محدودیتها و تجربههای مرتبط با استفاده عملی از مفاهیم.
- جمعبندی و سوالهای مرور :مرور مهمترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شدهاند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسانتر باشد.

این مجموعه بهصورت تدریجی و همزمان با انتشار جلسات جدید تکمیل میشود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش میدهد.

نسخههای بهروزشده جزوهها، منابع تکمیلی و یادداشتهای هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود. مخزن جزوهها:

https://github.com/LaneZero/data-ai-engineering-course

وبسایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثالهای افزودهشده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه میشود هنگام اجرای پروژههای واقعی، اطلاعات حساس یا تصمیمگیریهای فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، کمک به یادگیری عمیقتر، مرور منظم مفاهیم و اشتراک دانش با علاقهمندان حوزه داده و هوش مصنوعی است.

فهرست مطالب

۰.۱ اهداف یادگیری و محدوده جلسه ٤

۰.۲ چرا مهندس داده به لینوکس نیاز دارد؟ ٤

۰.۳ سیستم‌عامل، Kernel, Distribution و Shell ٤

۰.۴ فایل‌سیستم لینوکس و مسیرهای مهم ٥

۰.۵ ساختار دستورهای Linux ٦

۰.۶ جابه‌جایی در مسیرها و مشاهده محتوا ٦

۰.۷ ساخت، خواندن، کپی، انتقال و حذف فایل‌ها ۷

۰.۷.۱ تمرین عملی ایمن ۸

۰.۸ Pipe و اتصال دستورها ۸

۰.۹ تغییر مسیر خروجی و جست‌وجوی متن ۹

۰.۱۰ تاریخچه فرمان‌ها، میان‌برها و قطع اجرای برنامه ۹

۰.۱۱ حساب‌ها، دسترسی مدیریتی و نصب ابزار ۱۰

۰.۱۲ Shell Script و مباحثی که گذرا معرفی شدند ۱۰

۰.۱۳ چرا شناخت شبکه در مهندسی داده ضروری است؟ ۱۱

۰.۱۳.۱ MAC Address و IP Address ۱۱

۰.۱۳.۲ TCP و UDP ۱۱

۰.۱۳.۳ ادامه مبحث شبکه ۱۲

۰.۱۴ آشنایی مقدماتی با Git و GitHub ۱۲

۰.۱۵ تمرین‌ها و چک‌لیست جلسه ۱۲

۰.۱۶ جمع‌بندی ۱۳

۰.۱۷ واژه‌نامه جلسه ۱۳

۰.۱۸ سوال‌های مرور ١٤

۰.۱۹ پیوست: پاسخ‌نامه مختصر ۱۵

۰.۲۰ منابع و مطالعه بیشتر ١٦

۰.۱ اهداف یادگیری و محدوده جلسه

در این جلسه قرار هست پیش‌نیازهای کار فنی در مهندسی داده مرور شوند: آشنایی با لینوکس و Terminal، شناخت مقدماتی Git و GitHub و دریافت دید کلی از شبکه. این پیش‌نیازها هنگام کار روی سرور، اجرای برنامه، نگهداری سرویس‌ها، همکاری تیمی و اتصال ابزارهای داده به یکدیگر کاربرد دارند.

پس از مطالعه این جزوه باید بتوانید تفاوت Kernel و Distribution و Shell را توضیح دهید، در ساختار فایل‌های لینوکس جابه‌جا شوید، فایل و پوشه بسازید و مدیریت کنید، با Pipe و Redirection کار کنید، تاریخچه و راهنمای دستورات را ببینید و تفاوت کلی TCP و UDP را بشناسید.

۰.۲ چرا مهندس داده به لینوکس نیاز دارد؟

بخش زیادی از ابزارها و سرویس‌هایی که در مهندسی داده استفاده می‌شوند، روی سرورهای لینوکسی اجرا یا مدیریت می‌شوند. در محیط حرفه‌ای لازم است دست کم بتوانیم وارد یک Shell شویم، فایل‌ها و سرویس‌ها را بررسی کنیم، ابزارهای موردنیاز را نصب کنیم و خطاهای اولیه را بشناسیم. هدف این جلسه یادگیری کار با محیط گرافیکی دسکتاپ لینوکس نیست؛ تمرکز روی سرور و خط فرمان است.

برای تمرین هم‌زمان با کلاس، مدرس پیشنهاد کرد از محیط لینوکسی آماده، ماشین مجازی یا روش‌های مناسب برای کاربران ویندوز استفاده شود. کاربران macOS می‌توانستند بخشی از تمرین‌ها را با Terminal سیستم خود پیش ببرند. ایجاد حساب GitHub نیز از نیازمندی‌های پیشنهادی دوره بود.

> **تکمیل فنی:** Terminal در macOS و PowerShell در Windows دقیقاً معادل محیط GNU/Linux نیستند. برای خروجی‌های سازگارتر، محیط لینوکسی واقعی یا WSL در ویندوز مناسب‌تر است؛ بااین‌حال دستورات و امکانات دقیق به Shell و Distribution بستگی دارد.

۰.۳ سیستم‌عامل، Kernel, Distribution و Shell

سخت‌افزار شامل پردازنده، حافظه، دیسک و دستگاه‌های ورودی و خروجی است. سیستم‌عامل منابع را مدیریت می‌کند تا چند برنامه بتوانند هم‌زمان اجرا شوند. مدرس مثال زد که هنگام تماشای ویدیو، دانلود فایل و دریافت اعلان پیام‌رسان، سیستم‌عامل بدون آنکه کاربر پردازنده‌ها را شخصاً بین کارها تقسیم کند، اجرای برنامه‌ها را هماهنگ می‌کند.

Kernel (هسته): بخش مرکزی سیستم‌عامل که مسئول مدیریت منابع و ارتباط با سخت‌افزار است.

Distribution یا Distro (توزیع لینوکس): ترکیبی از هسته لینوکس و مجموعه‌ای از ابزارها، کتابخانه‌ها و تنظیمات؛ مانند Ubuntu و Arch Linux.

Shell (پوسته فرمان): محیطی که دستورهای کاربر را دریافت و اجرا می‌کند.

Terminal: برنامه‌ای که امکان تعامل با Shell را فراهم می‌کند.

در کلاس به تفاوت روش نصب بسته‌ها در توزیع‌های مختلف اشاره شد. برای نمونه، خانواده Debian/Ubuntu از ابزار `apt` استفاده می‌کند. رابط گرافیکی یا GUI برای کارهایی مانند بازکردن پنجره و کشیدن فایل مفید است، اما در مدیریت سرور اغلب Shell اهمیت بیشتری دارد.

> **تکمیل فنی:** «Kernel»، «Shell» و «Terminal» یک چیز نیستند. Terminal برنامه نمایش و دریافت ورودی است، Shell فرمان را تفسیر می‌کند و Kernel خدمات سطح پایین سیستم‌عامل را فراهم می‌کند. همچنین یک Distribution الزاماً فقط در Package Manager با دیگری تفاوت ندارد.

۰.۴ فایل‌سیستم لینوکس و مسیرهای مهم

برخلاف ساختار معمول درایوهای `C:` و `D:` در Windows، مسیرها در Linux از ریشه `/` شروع می‌شوند. مدرس چند مسیر پرکاربرد را به‌عنوان نقشه ذهنی معرفی کرد:

مسیر	کاربرد اصلی
`/`	ریشه فایل‌سیستم
`/home`	پوشه‌های خانگی کاربران عادی
`/root`	پوشه خانگی کاربر مدیریتی `root`
`/proc`	نمایش اطلاعات پردازه‌ها و اطلاعات سیستم به‌صورت فایل‌مانند
`/usr`	بسیاری از برنامه‌ها، کتابخانه‌ها و فایل‌های اشتراکی سیستم
`/etc`	بسیاری از فایل‌های پیکربندی سیستم و سرویس‌ها
`/var`	داده‌های متغیر مانند بخش‌هایی از لاگ‌ها و وضعیت سرویس‌ها
`/tmp`	فایل‌های موقت
`/dev`	رابط‌های فایل‌مانند مربوط به دستگاه‌ها

برای نمونه، هنگام اجرای برنامه یک **Process (پردازه)** ایجاد می‌شود و سیستم به آن شناسه می‌دهد. مسیر `/proc` برای مشاهده اطلاعات مربوط به پردازه‌ها و سیستم کاربرد دارد. در مثال‌های کلاس، نصب برنامه‌ها و محل پیکربندی یا ثبت لاگ نیز برای توضیح نقش مسیرها مطرح شد.

تکمیل فنی: `/proc` و بسیاری از ورودی‌های `/dev` فایل‌های معمول ذخیره‌شده روی دیسک نیستند؛ رابط‌هایی هستند که Kernel در اختیار برنامه‌ها می‌گذارد. محتوای `/tmp` نیز ممکن است در زمان راه‌اندازی مجدد یا طبق سیاست پاک‌سازی سیستم حذف شود، اما حذف خودکار همه فایل‌ها در هر راه‌اندازی مجدد تضمین عمومی ندارد.

۰.۵ ساختار دستورهای Linux

مدرس توضیح داد که معمولاً یک دستور از نام برنامه، گزینه‌ها یا Flagها و آرگومان‌ها تشکیل می‌شود. گزینه‌ها رفتار دستور را تغییر می‌دهند و آرگومان می‌تواند مسیر یا مقدار مورد پردازش باشد.

```
ls -la /home
```

در این مثال، `ls` نام دستور، `-l` و `-a` گزینه‌ها و `/home` مسیر مورد بررسی است. `-a` ورودی‌های مخفی را نیز نمایش می‌دهد و `-l` اطلاعات بیشتری مثل دسترسی‌ها و زمان تغییر را نشان می‌دهد. نام فایل‌های مخفی در لینوکس معمولاً با نقطه (`.`) شروع می‌شود.

تکمیل فنی: همه آرگومان‌ها الزامی نیستند. برای مثال `ls` بدون مسیر، محتوای مسیر کاری فعلی را نشان می‌دهد. ترتیب و نوع گزینه‌ها نیز به خود دستور بستگی دارد.

۰.۶ جابه‌جایی در مسیرها و مشاهده محتوا

دستور	نتیجه
`pwd`	نمایش مسیر کاری فعلی
`ls`	نمایش ورودی‌های مسیر فعلی
`ls -a`	نمایش ورودی‌های معمول و مخفی
`ls -la`	نمایش فهرست کامل‌تر
`ls /`	مشاهده ورودی‌های زیر ریشه
`cd /home`	جابه‌جایی به مسیر مشخص
`cd ..`	رفتن به پوشه والد

`cd .`	ماندن در همین مسیر
`cd`	بازگشت به پوشه خانگی کاربر
`clear`	پاک‌کردن نمای Terminal
`man ls`	مشاهده راهنمای دستور `ls` در صورت نصب بودن صفحات راهنما

مدرس نشان داد که یک دستور ممکن است در پوشه خالی چیزی چاپ نکند. تفاوتی میان «پوشه خالی است» و «دستور اجرا نشده» وجود دارد؛ برای تشخیص وضعیت باید مسیر فعلی و خروجی دستور را بررسی کرد.

```
pwd
ls -la
cd /tmp
pwd
cd ..
```

در کلاس همچنین به ابزار `tree` برای نمایش درختی پوشه‌ها اشاره شد. این ابزار ممکن است به‌صورت پیش‌فرض نصب نباشد.

۰.۷ ساخت، خواندن، کپی، انتقال و حذف فایل‌ها

در نمایش عملی، ساخت فایل، ساخت پوشه و تغییر محتوا مطرح شد. دستورهای کلیدی عبارت‌اند از:

دستور	کاربرد
`touch note.txt`	ایجاد فایل خالی در صورت نبودن یا تغییر زمان‌های ثبت‌شده فایل موجود
`mkdir practice`	ساخت پوشه
`cat note.txt`	نمایش محتوای فایل متنی
`cp source.txt copy.txt`	کپی فایل
`cp -r source_dir target_dir`	کپی بازگشتی پوشه
`mv old.txt new.txt`	انتقال یا تغییر نام فایل
`rm note.txt`	حذف فایل
`rm -r practice`	حذف بازگشتی پوشه و محتوای آن
`nano note.txt`	ویرایش متن با Nano در صورت نصب بودن
`vim note.txt`	ویرایش متن با Vim در صورت نصب بودن

مدرس نشان داد که می‌توان بدون ساخت قبلی فایل، آن را در یک ویرایشگر باز کرد و هنگام ذخیره، ایجادش کرد. استفاده از Tab برای تکمیل خودکار مسیرها و نام فایل‌ها نیز از نکات کاربردی بود.

تکمیل فنی: `rm -r` عملیات حذف بازگشتی انجام می‌دهد و ممکن است داده‌های مهم را بدون امکان بازیابی ساده از بین ببرد. پیش از اجرا، مسیر هدف را با `pwd` و `ls` بررسی کنید. در تمرین‌ها، حذف را فقط داخل پوشه آزمایشی خودتان انجام دهید و از اجرای دستورهای حذف روی مسیرهای سیستمی خودداری کنید.

۰.۷.۱ تمرین عملی ایمن

```
mkdir -p "$HOME/databurst-lab"
cd "$HOME/databurst-lab"
pwd
touch note.txt
printf 'Linux practice\n' > note.txt
cat note.txt
cp note.txt note-copy.txt
mv note-copy.txt renamed-note.txt
ls -la
```

مثال اضافی: این تمرین صرفاً برای تمرین شخصی طراحی شده و در Transcript به همین صورت اجرا نشده است. همه فایل‌ها در پوشه `databurst-lab` حساب خودتان ساخته می‌شوند و هیچ فایل سیستمی تغییر نمی‌کند.

۰.۸ Pipe و اتصال دستورها

یکی از ایده‌های اصلی مطرح‌شده در کلاس این است که ابزارهای خط فرمان معمولاً کارهای کوچک و مشخصی را انجام می‌دهند و می‌توان خروجی آن‌ها را به هم متصل کرد. عملگر Pipe یعنی `|` خروجی استاندارد یک فرمان را به ورودی استاندارد فرمان بعدی می‌فرستد.

```
printf 'Ali\nSara\nAli\nReza\n' | sort | uniq
```

در این مثال، `sort` ردیف‌ها را مرتب می‌کند و `uniq` ردیف‌های تکرارِ **مجاور** را حذف می‌کند. ترکیب این دو ابزار نتیجه‌ای مناسب برای حذف تکرارهای متنی ساده به دست می‌دهد.

تکمیل فنی: `uniq` به‌تنهایی تکرارهایی را که کنار هم نیستند حذف نمی‌کند. به همین دلیل، در نمونه عمومیِ حذف مقادیر تکراری، معمولاً قبل از آن `sort` قرار می‌گیرد.

۰.۹ تغییر مسیر خروجی و جست‌وجوی متن

مدرس تفاوت نوشتن خروجی در Terminal و فرستادن آن به فایل را نشان داد. عملگر `>` محتوا را در فایل مقصد می‌نویسد و اگر فایل موجود باشد، محتوای قبلی آن را جایگزین می‌کند. عملگر `>>` خروجی را به انتهای فایل اضافه می‌کند.

```
printf 'first\n' > sample.txt
printf 'second\n' >> sample.txt
cat sample.txt
```

برای یافتن خط‌هایی که عبارت مشخصی دارند، نمونه‌ای از جست‌وجوی متن در فایل‌های لاگ مطرح شد. ابزار `grep` برای چنین کارهایی مناسب است:

```
printf 'login ok\nlogout ok\nlogin failed\n' > app.log
grep 'login' app.log
```

تکمیل فنی: در Shell، خروجی استاندارد (`stdout`) و خروجی خطا (`stderr`) دو جریان جدا هستند. `>` به‌صورت پیش‌فرض خروجی استاندارد را هدایت می‌کند و الزاماً خطاها را در همان فایل نمی‌نویسد. همچنین این عملگر می‌تواند محتوای فایل موجود را بازنویسی کند؛ قبل از استفاده مسیر فایل را بررسی کنید.

۰.۱۰ تاریخچه فرمان‌ها، میان‌برها و قطع اجرای برنامه

مدرس چند قابلیت ساده برای افزایش سرعت کار با Terminal معرفی کرد:

کلید یا دستور	کاربرد
`↑` و `↓`	پیمایش دستورهای قبلی و بعدی در تاریخچه
`Ctrl + R`	جست‌وجوی معکوس در تاریخچه Shellهای پشتیبانی‌کننده
`history`	نمایش تاریخچه فرمان‌ها در Shellهای متداول
`Tab`	تکمیل خودکار فرمان یا مسیر، بسته به Shell
`Ctrl + C`	ارسال سیگنال قطع به برنامه پیش‌زمینه
`sleep 20`	مکث به مدت ۲۰ ثانیه برای تمرین رفتار فرمان طولانی

در دمو، مدرس فرمانی شبیه `sleep` را با مدت طولانی اجرا کرد و نشان داد با `Ctrl + C` می‌توان اجرای آن را متوقف کرد.

تکمیل فنی: `Ctrl + C` معمولاً سیگنال `SIGINT` ارسال می‌کند. بیشتر برنامه‌های تعاملی با آن متوقف می‌شوند، اما هر برنامه می‌تواند این سیگنال را متفاوت مدیریت کند؛ بنابراین «همیشه برنامه را می‌بندد» قاعده قطعی نیست.

۰.۱۱ حساب‌ها، دسترسی مدیریتی و نصب ابزار

در Linux، کاربر `root` دسترسی مدیریتی گسترده دارد. مدرس توضیح داد که روی یک سرور مشترک ممکن است چند نفر نیاز به انجام برخی کارهای مدیریتی داشته باشند؛ در این شرایط می‌توان با تنظیم مجوزها، اجرای فرمان‌های معینی را از طریق `sudo` ممکن کرد. برای نمونه، مدیریت یک سرویس مانند Kafka نیازمند دسترسی مناسب است.

در بخش نصب ابزارها، به Package Manager و تفاوت روش‌های نصب در Linux, macOS و Windows اشاره شد. برای نمونه، در توزیع‌های مبتنی بر Debian/Ubuntu نصب ابزارها معمولاً با `apt` انجام می‌شود و در macOS از ابزارهایی مانند Homebrew می‌توان استفاده کرد.

تکمیل فنی: `sudo` به معنای «مجازبودن اجرای هر فرمان با دسترسی root» برای همه کاربران نیست؛ سیاست آن می‌تواند محدود به فرمان‌های مشخص باشد. اجرای کورکورانه دستورهای اینترنتی با `sudo` ریسک جدی دارد و قبل از نصب باید منبع و عملکرد فرمان بررسی شود.

۰.۱۲ Shell Script و مباحثی که گذرا معرفی شدند

مدرس توضیح داد که مجموعه‌ای از فرمان‌ها را می‌توان در یک فایل متنی ذخیره کرد و مانند یک برنامه اجرا کرد؛ به این فایل **Shell Script** می‌گویند. به موضوع مجوز اجرا نیز اشاره شد. مباحثی مانند دسترسی خواندن و نوشتن فایل‌ها، Linkها و جزئیات بیشتر مدیریت سیستم فقط گذرا مطرح شدند و برای مطالعه بیشتر باقی ماندند.

```
#!/usr/bin/env bash
# Simple practice script
printf 'Hello from the shell\n'
touch "$HOME/databurst-script-created.txt"
```

مثال اضافی: این نمونه، شکل ساده یک Script است و نقل دقیق Demo کلاس نیست. در یک فایل مانند `hello.sh` ذخیره می‌شود و می‌توان آن را با `bash hello.sh` اجرا کرد؛ نمونه فقط یک پیام چاپ می‌کند و یک فایل در پوشه خانگی می‌سازد.

۰.۱۳ چرا شناخت شبکه در مهندسی داده ضروری است؟

در ادامه جلسه، مدرس به این مسئله پرداخت که هنگام وصل‌شدن برنامه‌ها و سرویس‌های مختلف، داده از یک ماشین به ماشین دیگر چگونه منتقل می‌شود. برای نمونه، اتصال به Kafka یا استقرار یک برنامه روی سرور، بدون شناخت اولیه شبکه ممکن است دشوار شود.

از مدل لایه‌ای شبکه صحبت شد و به این ایده اشاره شد که هر لایه مسئله خاصی را حل می‌کند. لایه فیزیکی با انتقال بیت‌ها سروکار دارد و لایه‌های دیگر وظایفی مانند آدرس‌دهی، انتقال و ارتباط برنامه‌ها را پوشش می‌دهند.

۰.۱۳.۱ MAC Address و IP Address

مدرس برای توضیح شناسایی دستگاه‌ها، از **MAC Address** در ارتباط محلی و **IP Address** در آدرس‌دهی شبکه صحبت کرد. همچنین اشاره کرد که یک سرور ممکن است چند رابط شبکه داشته باشد و از چند مسیر به شبکه‌های مختلف متصل شود.

> **تکمیل فنی:** MAC Address معمولاً شناسه یک رابط در شبکه محلی است، نه شناسه ثابت و جهانیِ تمام ارتباطات دستگاه. IP Address نیز به رابط یا تنظیم اتصال در یک شبکه مربوط است و ممکن است تغییر کند. برای شناخت ارتباط بین شبکه‌های مختلف باید نقش Router و مسیریابی را هم در نظر گرفت.

۰.۱۳.۲ TCP و UDP

مدرس با مثال درخواست بازشدن یک وب‌سایت و مثال پخش زنده یا تماس صوتی، تفاوت دو شیوه انتقال را مطرح کرد. در توضیح مفهومی، **TCP** برای ارتباطی به‌کار می‌رود که تحویل و ترتیب داده‌ها اهمیت دارد؛ در مقابل **UDP** سربار کمتری دارد و برای برخی ارتباط‌های بلادرنگ یا حساس به تأخیر مناسب است.

ویژگی	TCP	UDP
نوع ارتباط	اتصال‌گرا	بدون ایجاد اتصال در سطح پروتکل
تحویل و ترتیب در سطح پروتکل	دارای سازوکار اطمینان از تحویل و تحویل مرتب بایت‌ها	تحویل و ترتیب را تضمین نمی‌کند
سربار	معمولاً بیشتر	معمولاً کمتر
مثال مفهومی	بسیاری از ارتباط‌های مبتنی بر TCP	برخی کاربردهای صوتی، تصویری و زمان‌حساس

تکمیل فنی: TCP تضمین نمی‌کند که هر ارتباطی حتماً در نهایت موفق شود؛ در خطاهای پایدار، اتصال می‌تواند قطع شود. همچنین همه ترافیک ویدیویی مبتنی بر UDP نیست و بعضی برنامه‌ها روی UDP سازوکارهای اطمینان و کنترل ازدحام خود را پیاده می‌کنند. مثال‌های کلاس برای ایجاد شهود هستند، نه تعیین قطعی پروتکل همه سرویس‌ها.

۰.۱۳.۳ ادامه مبحث شبکه

در پایان، مدرس گفت که زمان جلسه اجازه ورود کامل به بخش شبکه را نمی‌دهد و قرار شد موضوعات باقیمانده در جلسه بعد بررسی شوند. بنابراین مفاهیم پیشرفته‌تر شبکه، TLS و جزئیات ارتباط سرویس‌ها در این جزوه به‌عنوان مباحث تدریس‌شده کامل ارائه نشده‌اند.

۰.۱۴ آشنایی مقدماتی با Git و GitHub

Git به‌عنوان سامانه‌ای برای **Version Control** معرفی شد: روشی برای نگهداری تاریخچه تغییرات و روشن‌کردن اینکه چه تغییری، در چه مرحله‌ای و با چه هدفی انجام شده است. GitHub نیز به‌عنوان پلتفرمی برای میزبانی مخزن‌ها و همکاری معرفی شد.

در کلاس توضیح داده شد که وقتی افراد زیادی روی یک پروژه کار می‌کنند، نگهداری تاریخچه صرفاً روی لپ‌تاپ یک نفر کافی نیست. داشتن مخزن قابل‌دسترسی برای همکاران، ادامه کار و بازبینی تغییرات را آسان‌تر می‌کند. مدرس ساخت حساب GitHub را پیشنهاد کرد و برای یادگیری تکمیلی Git به دوره رایگان جادی اشاره کرد.

تکمیل فنی: Git و GitHub یکی نیستند: Git ابزار مدیریت نسخه است و می‌تواند کاملاً محلی کار کند؛ GitHub یکی از سرویس‌های میزبانی مخزن Git است. آموزش عملی دستورهای `git init`، `git add`، `git commit` و `git push` در این Transcript به‌طور کامل ثبت نشده و در این جزوه به مدرس نسبت داده نمی‌شود.

۰.۱۵ تمرین‌ها و چک‌لیست جلسه

این تمرین‌ها بر مبنای دستورهای تدریس‌شده و با مسیرهای ایمن تنظیم شده‌اند:

- [ ] یک محیط Linux مناسب برای تمرین آماده کنید و `pwd` را اجرا کنید.
- [ ] با `ls -la` فایل‌های معمول و مخفی مسیر فعلی را ببینید.

- [ ] با `cd` و `cd ..` بین مسیرها جابه‌جا شوید.
- [ ] پوشه آزمایشی بسازید و در آن با `touch` یک فایل ایجاد کنید.
- [ ] فایل را با `cat` ببینید، با `cp` کپی کنید و با `mv` تغییر نام دهید.
- [ ] با `sort | uniq` مقادیر تکراری یک فهرست متنی را حذف کنید.
- [ ] فرق `>` و `>>` را در یک فایل آزمایشی مشاهده کنید.
- [ ] برای مشاهده راهنمای یک دستور، `man` یا راهنمای همان ابزار را بررسی کنید.
- [ ] با `history` و `Ctrl + R` یک فرمان قبلی را پیدا کنید.
- [ ] حساب GitHub خود را آماده کنید و تفاوت Git و GitHub را توضیح دهید.
- [ ] فرق کلی MAC, IP, TCP و UDP را به زبان خودتان بازگو کنید.

۰.۱۶ جمع‌بندی

هسته اصلی این جلسه آشنایی با محیطی بود که مهندس داده در عمل زیاد با آن روبه‌رو می‌شود: Linux Shell. مسیرهای اصلی فایل‌سیستم و فرمان‌های پایه برای مشاهده، ویرایش و جابه‌جایی فایل‌ها مرور شدند. سپس مشخص شد چگونه می‌توان ابزارهای کوچک را با Pipe و Redirection به یکدیگر متصل کرد و از تاریخچه، راهنما و میان‌برها کمک گرفت. در پایان، Git و GitHub به‌عنوان پایه همکاری تیمی معرفی شدند و مقدمات شبکه برای فهم ارتباط سرویس‌ها مطرح شد. **ادامه شبکه و آموزش عملی کامل Git جزو مطالب تکمیل‌شده این جلسه نبودند.**

۰.۱۷ واژه‌نامه جلسه

ردیف	اصطلاح انگلیسی	معادل یا توضیح فارسی
۱	Linux	خانواده سیستم‌عامل‌های مبتنی بر هسته لینوکس
۲	Kernel	هسته سیستم‌عامل
۳	Distribution (Distro)	توزیع لینوکس
۴	Shell	پوسته اجرای فرمان
۵	Terminal	محیط تعامل متنی با Shell
۶	CLI	رابط خط فرمان
۷	GUI	رابط کاربری گرافیکی
۸	Process	پردازه

۹	File System	فایل‌سیستم
۱۰	Package Manager	مدیر بسته نرم‌افزاری
۱۱	Argument	آرگومان یا ورودی فرمان
۱۲	Option / Flag	گزینه تغییر رفتار فرمان
۱۳	Pipe	اتصال خروجی یک فرمان به ورودی فرمان دیگر
۱۴	Redirection	تغییر مسیر ورودی یا خروجی
۱۵	Standard Output (stdout)	خروجی استاندارد
۱۶	Standard Error (stderr)	خروجی خطای استاندارد
۱۷	Shell Script	فایل شامل فرمان‌های Shell
۱۸	sudo	اجرای فرمان با سطح دسترسی مجازِ دیگر
۱۹	Git	سامانه مدیریت نسخه توزیع‌شده
۲۰	GitHub	سرویس میزبانی و همکاری روی مخزن Git
۲۱	MAC Address	شناسه لایه پیوند داده برای رابط شبکه
۲۲	IP Address	آدرس منطقی در شبکه IP
۲۳	TCP	پروتکل انتقال اتصال‌گرا
۲۴	UDP	پروتکل انتقال بدون اتصال

۰.۱۸ سوال‌های مرور

۱. چرا کار با Linux و Shell برای بسیاری از نقش‌های مهندسی داده اهمیت دارد؟

۲. تفاوت Kernel, Distribution, Terminal و Shell چیست؟

۳. تفاوت `/home` و `/root` را توضیح دهید.

۴. مسیر `/proc` چه اطلاعاتی در اختیار ما قرار می‌دهد؟

۵. تفاوت خروجی `ls` و `ls -la` چیست؟

۶. دستور `cd ..` چه کاری انجام می‌دهد و با `cd` بدون آرگومان چه فرقی دارد؟

۷. تفاوت `cp`، `mv` و `rm` در مدیریت فایل چیست؟

۸. Pipe چه کاربردی دارد و چرا در مثال حذف تکرار، `sort` را پیش از `uniq` می‌آوریم؟

۹. تفاوت `>` و `>>` چیست و چرا باید در استفاده از آن‌ها دقت کرد؟

۱۰. برای پیدا کردن فرمانی که قبلاً اجرا کرده‌اید، چه راه‌هایی دارید؟

۱۱. `Ctrl + C` معمولاً چه اتفاقی را در Terminal رقم می‌زند؟

۱۲. `sudo` چه مسئله‌ای را حل می‌کند و چه ریسکی دارد؟

۱۳. Git با GitHub چه تفاوتی دارد؟

۱۴. MAC Address و IP Address هر کدام در چه سطحی از ارتباط شبکه کاربرد دارند؟

۱۵. تفاوت کلی TCP و UDP چیست و چرا ممکن است برای یک برنامه بلادرنگ UDP انتخاب شود؟

۰.۱۹ پیوست: پاسخ‌نامه مختصر

۱. چون بسیاری از ابزارهای داده و سرویس‌های سروری در محیط‌های لینوکسی اجرا می‌شوند و نیازمند کار با خط فرمان‌اند.

۲. Kernel منابع سیستم را مدیریت می‌کند؛ Distribution مجموعه‌ای مبتنی بر لینوکس است؛ Terminal محیط تعامل است؛ Shell فرمان‌ها را اجرا می‌کند.

۳. `/home` معمولاً پوشه‌های خانگی کاربران عادی را در خود دارد و `/root` پوشه خانگی کاربر root است.

۴. اطلاعات پردازه‌ها و بخش‌هایی از وضعیت سیستم، عمدتاً از طریق یک فایل‌سیستم مجازی.

۵. `ls -la` جزئیات بیشتر و ورودی‌های مخفی را هم نشان می‌دهد.

۶. `cd ..` به پوشه والد می‌رود؛ `cd` معمولاً به پوشه خانگی کاربر برمی‌گردد.

۷. `cp` کپی می‌کند؛ `mv` انتقال یا تغییر نام می‌دهد؛ `rm` حذف می‌کند.

۸. Pipe جریان خروجی یک دستور را به ورودی دیگری می‌دهد؛ `uniq` فقط تکرارهای مجاور را حذف می‌کند.

فایل را ایجاد یا بازنویسی می‌کند و `>>` خروجی را به انتهای آن می‌افزاید.

۹. کلیدهای بالا/پایین، `history` و جست‌وجوی تاریخچه با `Ctrl + R`.

۱۰. معمولاً سیگنال SIGINT به برنامه پیش‌زمینه می‌فرستد.

۱۱. اجرای فرمان‌ها با مجوزهای تعریف‌شده را ممکن می‌کند، اما در صورت استفاده نادرست می‌تواند به سیستم آسیب بزند.

۱۲. Git سیستم مدیریت نسخه است؛ GitHub سرویس میزبانی و همکاری است.

۱۳. MAC برای ارتباط لایه پیوند داده در شبکه محلی و IP برای آدرس‌دهی و مسیریابی در شبکه‌های به کار می‌رود.

۱۴. TCP سازوکار تحویل و ترتیب دارد؛ UDP چنین تضمین‌هایی در سطح پروتکل ندارد و می‌تواند برای برخی کاربردهای کم‌تأخیر مناسب باشد.

۰.۲۰ منابع و مطالعه بیشتر

دوره رایگان جادی : آمادگی آزمون | LPIC-1 مکتب‌خونه